

Queues

A queue is a data structure that stores elements using First-In-First-Out (FIFO) order: the first element added is the first to be removed.

Natural Explanation

Think of a line at a checkout: the first person to arrive is the first to be served. New arrivals join the back; the person at the front leaves first.

Core Operations

ENQUEUE(x): Add element **x** to the back of the queue.

DEQUEUE(): Remove and return the element at the front.

FRONT() / **PEEK()**: Return the front element without removing it.

ISEMPTY(): Check if the queue contains no elements.

All operations run in $O(1)$ amortized time when using a circular array, or $O(1)$ in all cases with a linked list.

Typical Implementation

With a dynamic array (circular): - Maintain indices **front** and **back** - ENQUEUE adds to **back** and advances it - DEQUEUE removes from **front** and advances it - When either index reaches the end, wrap around (modular arithmetic) - Avoids wasteful shifts of elements

With a linked list: - Keep pointers to head (front) and tail (back) - ENQUEUE appends to tail - DEQUEUE removes from head

Common Uses

- **Breadth-first search**: exploring vertices by distance
 - **Level-order tree traversal**: processing nodes level by level
 - **CPU scheduling**: task queues in operating systems
 - **Message passing**: buffering tasks or events
 - **Load balancing**: distributing work fairly (FIFO is fair)
-

FIFO Property

Access is restricted to the front. You cannot directly access the middle without removing elements first.

Related

- [\[\[Stacks\]\]](#)
- [\[\[Linked Lists\]\]](#)
- [\[\[Breadth-First Search\]\]](#)
- [\[\[Priority Queues\]\]](#)